

16

FINDING FRAUDS WITH BENFORD'S LAW



Prior to the invention of electronic calculators, if you needed to take the log of a number, you looked it up in a table. Astronomer Simon Newcomb used such tables, and in 1881, he noticed that the pages in the front, used for numbers beginning with the lowest digits, were more worn than those in the back. From this mundane observation, he realized that—at least for measurements and constants in nature—the leading digits were much more likely to be small than large. He published a short article about it and moved on.

For decades, this statistical curiosity, like Tolkien's One Ring, "passed out of all knowledge." Then, in 1938, physicist Frank Benford rediscovered and confirmed the phenomenon by collecting over 20,000 samples of real-world numbers, using data sources as diverse as the measurements of rivers, street addresses, numbers contained in *Reader's Digest* magazine, molecular weights, baseball statistics, death rates, and more. As the person who popularized this scientific discovery, he got all the credit.

According to *Benford's law*, also known as the *first-digit law*, the frequency of occurrence of the leading digits in naturally occurring numerical distributions is predictable and nonuniform. In fact, a given number is six times more likely to start with a 1 than a 9! This is very counterintuitive, as most people would expect a uniform distribution, with each num-

ber having a one in nine (11.1 percent) chance of occupying the first slot. Due to this cognitive disconnect, Benford's law has become a useful tool for fraud detection in financial, scientific, and election data.

In this chapter, you'll write a Python program that compares real-life datasets to Benford's law and determines whether they appear fraudulent or not. You'll also dust off `matplotlib` one last time to add a useful visual component to the analysis. For a dataset, you'll use votes cast in the contentious 2016 US presidential election.

Project #24: Benford's Law of Leading Digits

Figure 16-1 depicts a bar chart of the leading significant digits in a set of numbers following Benford's law. Surprisingly, scale doesn't matter. A tabulation of the length of Australian roads will follow Benford's law whether it is in miles, kilometers, or cubits! As a statistical principle, it is *scale invariant*.

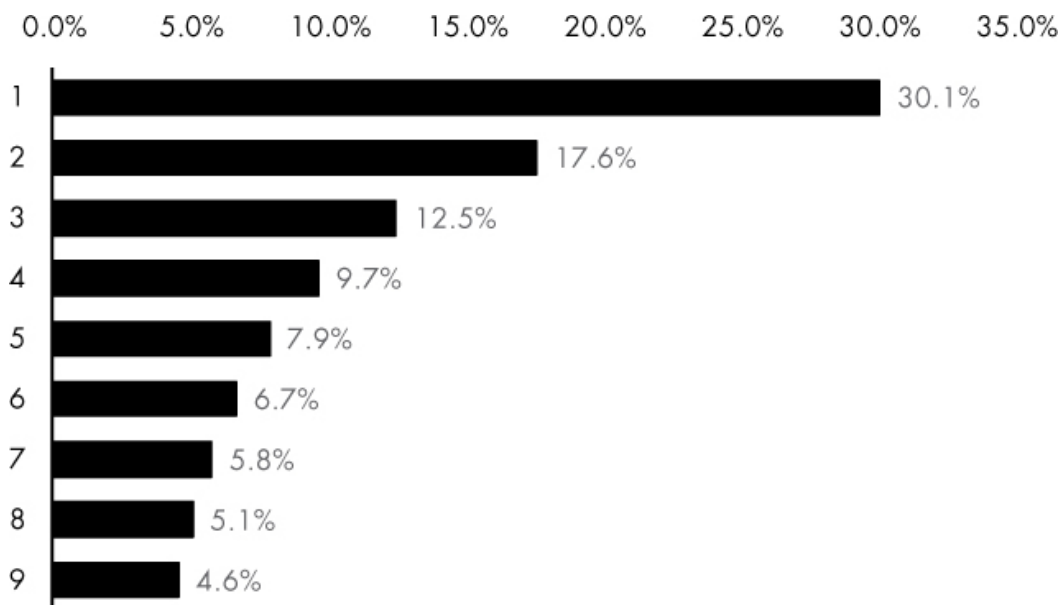


Figure 16-1: The frequency of occurrence of leading digits according to Benford's law

It took mathematicians about a hundred years to come up with an explanation for Benford's law that *they* found satisfactory. For the rest of us, let's just say that there are more little things in the universe than large things. Frank Benford used the analogy that owning *one* acre of land is easier than owning *nine* acres. In fact, you can closely duplicate the frequency produced by Benford's law by simply assuming that there are

twice as many 1s as 2s, three times as many 1s as 3s, and so on. You just take the inverse of each of the nine digits ($1/d$) and divide by the sum of all the inverses (2.83). Then multiply the results by 100 to get the percentage (see **Figure 16-2**).

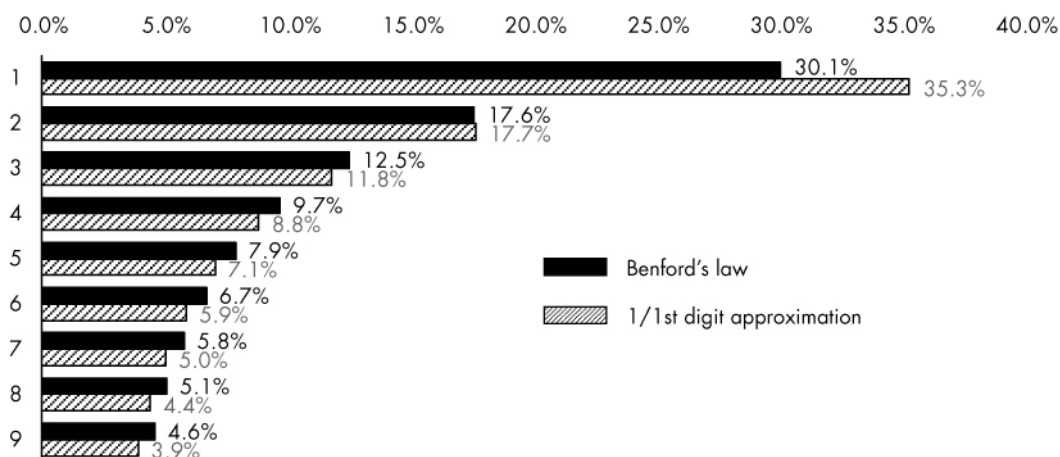


Figure 16-2: Comparison of Benford's law with an approximation proportional to the inverse of the first digit

Because of the size relationships just discussed, Benford's law can be visualized with a *logarithmic* scale, which is used to graph data that are related according to an exponential relationship. In *semilogarithmic* ("semilog") plots, one variable tends to be restricted, like the set of leading digits (1–9), while the other covers a wide range of values that includes several orders of magnitude.

On semilog graph paper, the horizontal x-axis values are log values, and the vertical y-axis values, represented by horizontal lines, are not (see **Figure 16-3**). On the x-axis, the horizontal divisions aren't regular, and this nonlinear pattern repeats with powers of 10. For every *decade* on the log paper, such as 1 through 10 or 10 through 100, the *width* of the divisions between numbers is proportional to the *length* of the bars in **Figure 16-1**. For example, the distance between the 1 and 2 in **Figure 16-3** is 30.1 percent of the distance between 1 and 10. As one author put it, you could derive Benford's law by simply throwing darts at a piece of log paper!



Figure 16-3: Example of two-decade semilogarithmic graph paper

For a collection of numeric data to conform to Benford’s law, certain conditions have to be met. The numbers need to be random and not assigned, with no imposed minimums or maximums. The numbers should cover several orders of magnitude, and the dataset should be large; recommendations in the literature call for 100 to 1,000 samples as a minimum, though Benford’s law has been shown to hold true for datasets containing as few as 50 numbers. Examples of distributions that won’t follow Benford’s law are the heights of professional basketball players, US telephone numbers (for which only the last four digits are truly random), prices influenced by psychological barriers (\$1.99 versus \$2.00), and medical copayments.

Applying Benford’s Law

Most financial and accounting data conform to naturally occurring numbers and are thus subject to Benford’s law. For example, assume you own a stock mutual fund with a value of \$1,000. For your fund to reach \$2,000 in value, it would have to double by growing 100 percent. To increase from \$2,000 to \$3,000, it would only need to grow by 50 percent. For the first digit to be a 4, it needs to grow by another 33 percent. As Benford’s law predicts, for the leading digit 1 to become 2, there needs to be more growth than for the 3 to become 4, and so on. Because a Benford distribution is a “distribution of distributions,” financial datasets tend to conform, as they result from combining numbers—though exceptions do occur.

Because people are generally unaware of Benford’s law, they don’t account for it when falsifying numerical records. This gives forensic accountants a powerful tool for quickly identifying datasets that may be fraudulent. In fact, comparisons to Benford’s law are legally admissible as evidence in US criminal cases at the federal, state, and local levels.

In the 1993 case of *State of Arizona v. Nelson*, the accused diverted nearly \$2 million to bogus vendors in an attempt to defraud the state. Despite taking care to make the fake checks look legitimate, the first-digit distribution clearly violated Benford's law (**Figure 16-4**), leading to a conviction.

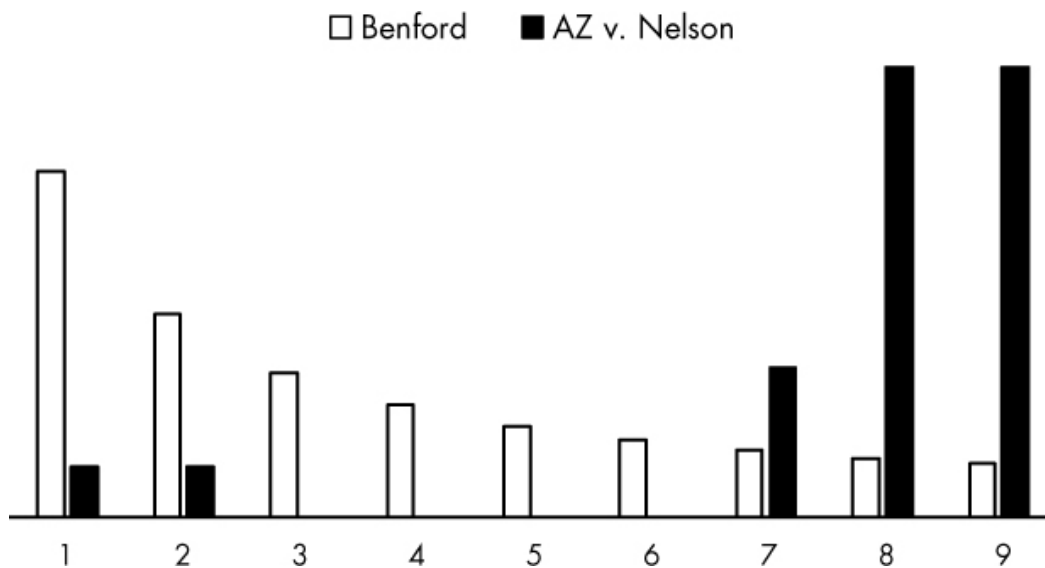


Figure 16-4: Comparison of the frequency of first digits in fraudulent checks to anticipated Benford's law frequency, State of Arizona v. Wayne James Nelson (CV92-18841)

Benford's law is also useful for internal business audits. Imagine a case where any travel and entertainment expenses over \$10,000 must be approved by the company vice president. This type of financial threshold can tempt employees to do things like split invoices to game the system.

Figure 16-5 is based on a group of expenses ranging from \$100 to \$12,000, where all values greater than \$9,999 were split into two equal halves. As you can guess, there is a spike in first-digit frequencies around 5 and 6, in clear violation of Benford's law.

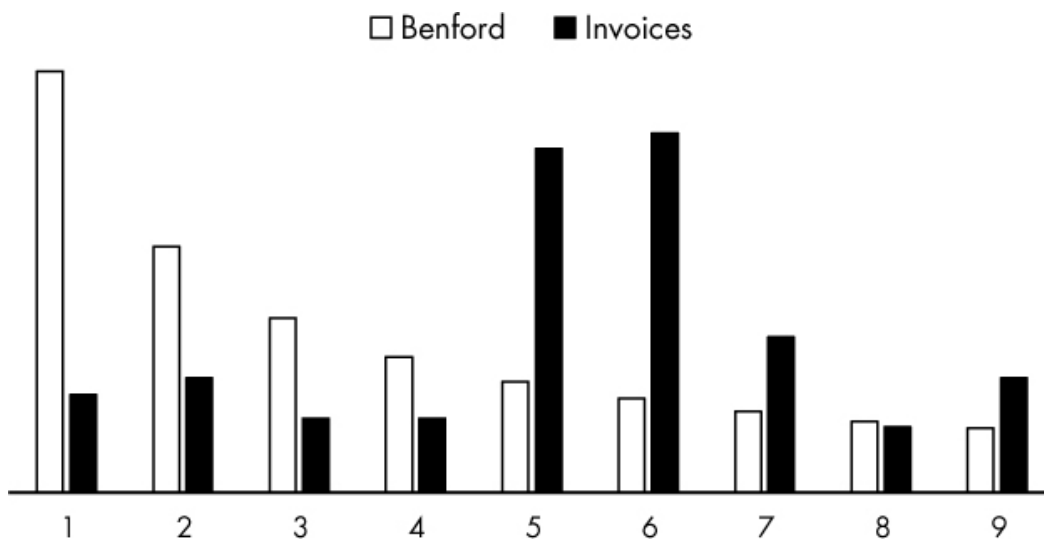


Figure 16-5: Splitting invoices over \$9,999 for a dataset ranging from \$100 to \$12,000 violates Benford's law.

On a grander scale, Benford's law has revealed irregularities in the financial data—such as revenue numbers—of large corporations. An example from Enron, which practiced institutionalized accounting fraud, is shown in **Figure 16-6**. Enron's bankruptcy in 2001 was the largest in history at the time, and it resulted in the jailing of several top executives. The scandal also brought about the dissolution of Arthur Andersen, a “Big Five” accounting firm and one of the world's largest multinational corporations.

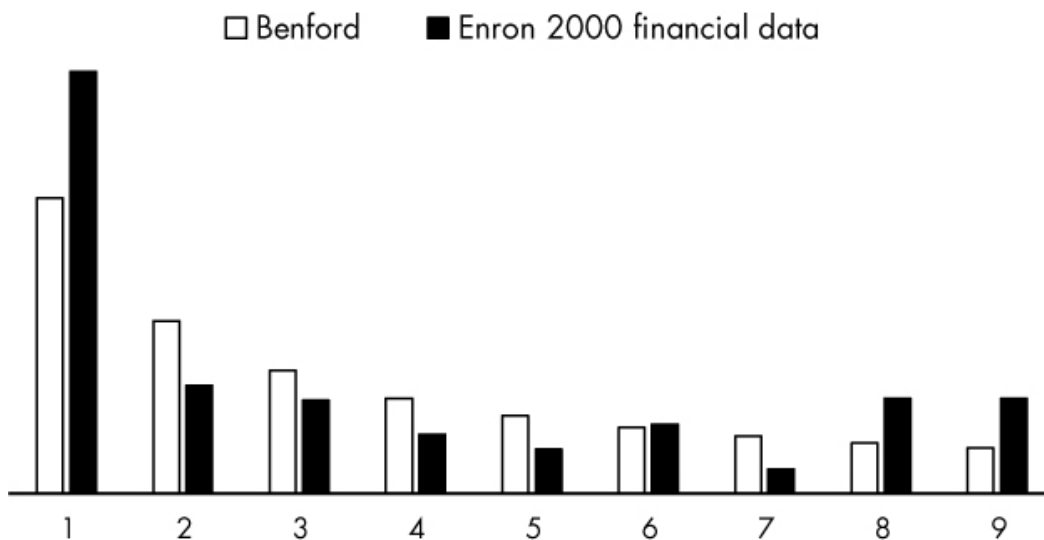


Figure 16-6: Frequency of first digits from Enron's year 2000 financial data versus anticipated frequency based on Benford's law (after the Wall Street Journal)

Obviously, Benford's law works best for fraud detection when the criminals are unaware of it. If you know how the law works, you can fool it,

which we'll do in a practice project at the end of this chapter. As a result, you can use Benford's law to flag datasets that might be fraudulent, but you can't use it to prove the opposite.

Performing the Chi-Square Test

Auditors and investigators use multiple statistical methods to verify that a dataset obeys Benford's law. In this project, you'll use the *chi-square goodness-of-fit test*, which is a commonly used method for determining whether an empirical (observed) distribution differs significantly from a theoretical (expected) distribution. A significance level, or *p-value*, is used as the discriminator. The most common significance level is 0.05, but other common ones include 0.01 and 0.10. A significance level of 0.05 indicates a 5 percent risk of erroneously concluding that a difference exists when it doesn't.

Here are the steps for performing a chi-square goodness-of-fit test:

1. Find the *degrees of freedom (df)*, defined as the number of categories (*k*) minus 1:

$$df = k - 1$$

For Benford's law, the categorical levels are the first digits (1–9), so $df = 8$.

2. Calculate the expected frequency count at each level by multiplying the sample size by the theoretical proportions at each level:

$$E_i = np_i$$

where E is the expected frequency at the i th level, n is the sample size, and p is the theoretical probability at the i th level. For 1,000 samples, the number of samples expected to start with 1 in a Benford's law distribution would be $1,000 \times 0.301 = 301$ (see **Figure 16-1**).

3. Calculate the chi-square random variable (X^2), also known as the *test statistic*, which will allow you to judge whether the two distributions are the same:

$$X_{df}^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

where O is the observed frequency count for the i th level of the categorical variable, E is the expected frequency count for the i th level of the categorical variable, and df represents the *degrees of freedom*.

4. Refer to the *chi-square distribution table* (**Table 16-1**), reading across the row corresponding to the calculated degrees of freedom. If the test statistic is *less than* the value shown in the *p*-value column considered significant, then you can't reject the hypothesis that the observed and theoretical distributions are the same.

Table 16-1: Chi-Square Distribution Table

Degrees of freedom	Probability of exceeding the critical value								
	0.99	0.95	0.90	0.75	0.50	0.25	0.10	0.05	0.01
1	0.000	0.004	0.016	0.102	0.455	1.32	2.71	3.84	6.63
2	0.020	0.103	0.211	0.575	1.386	2.77	4.61	5.99	9.21
3	0.115	0.352	0.584	1.212	2.366	4.11	6.25	7.81	11.34
4	0.297	0.711	1.064	1.923	3.357	5.39	7.78	9.49	13.28
5	0.554	1.145	1.610	2.675	4.351	6.63	9.24	11.07	15.09
6	0.872	1.635	2.204	3.455	5.348	7.84	10.64	12.59	16.81
7	1.239	2.167	2.833	4.255	6.346	9.04	12.02	14.07	18.48
8	1.647	2.733	3.490	5.071	7.344	10.22	13.36	15.51	20.09
9	2.088	3.325	4.168	5.899	8.343	11.39	14.68	16.92	21.67
10	2.558	3.940	4.865	6.737	9.342	12.55	15.99	18.31	23.21
Not significant					Significant				

In Table 16-2, the critical value for a *p*-value of 0.05—at 8 degrees of freedom—is 15.51. If your calculated test statistic is less than 15.51, then the corresponding *p*-value is greater than 0.05, and you would conclude that

there is *no statistically significant difference* between the observed distribution and the one predicted by Benford's law. The p -value here is the probability that a test statistic having 8 degrees of freedom is more extreme than 15.51.

Note that you should perform the chi-square test on *counts*. If your data is in percentages, averages, ratios, or so on, then you'll need to convert the values to counts before running the test.

THE OBJECTIVE

Write a Python program that loads numerical data, records the frequency of occurrence of the first digits, compares these to Benford's law using the chi-square goodness-of-fit test, and presents the comparison in both tabular and graphical form.

The Dataset

The 2016 US presidential election was rife with allegations of voter fraud. Most famously, the Russians were accused of supporting Donald Trump, and the Democratic National Committee was accused of favoring Hillary Clinton over Bernie Sanders in that party's nomination process. President Trump also alleged that 5 to 6 million people voted illegally, and in May 2017, he signed an executive order that launched a commission to review voter fraud and voter suppression.

For this project, you'll use a dataset of voting records from the 2016 presidential election. This consists of the final *by-county* votes for the 102 counties in the state of Illinois, which was won by Hillary Clinton. Starting in June 2016, the Illinois Voter Registration System database was the victim of a malicious cyberattack of unknown origin. Illinois election officials confirmed that hackers had accessed thousands of records but apparently didn't alter any data.

There were a surprising number of candidates on the Illinois ballot for president, so the dataset has been parsed to include only Hillary Clinton,

Donald Trump, Gary Johnson, and Jill Stein. Votes for these candidates have been lumped together in a 408-line text file, with these first 5 lines:

```
962
997
1020
1025
1031
```

You can find the complete tally of candidates and votes online at <https://www.elections.il.gov/ElectionInformation/DownloadVoteTotals.aspx>.

For this project, you need only the votes, which can be downloaded from <https://www.nostarch.com/impracticalpython/> as *Illinois_votes.txt*. You'll need to keep this file in the same folder as your Python code.

The Strategy

Let's pretend you're an investigator looking into allegations of voter fraud in the 2016 presidential election and you've been assigned the state of Illinois. Before doing a deep dive into the data, you'll want to flag any obvious anomalies. Benford's law can't help you determine whether people are voting illegally, but it's a good place to start for detecting vote *tampering*—that is, changing votes after they are cast.

In cases like this, the ability to communicate the results is every bit as important as the quantitative analysis. Voting commissions don't just include experts but also many laypeople with limited knowledge of statistics. And juries probably will not include a single expert. To convince yourself—and others—that the vote counts are valid (or not), you'll want to show multiple comparisons, like a table, a graph, and the quantitative chi-square variable (test statistic).

The individual steps involved in the analysis lend themselves well to encapsulation in functions. So, instead of pseudocode, let's look at what functions you may need:

`load_data()` Load the data as a list.

`count_first_digits()` Tabulate the first digits in each county's observed vote total.

`get_expected_counts()` Determine the counts for each first digit as predicted by Benford.

`chi_square_test()` Run a chi-square goodness-of-fit test on the observed versus expected counts.

`bar_chart()` Generate a bar chart to compare observed first-digit percentages to those expected.

`main()` Get the dataset filename, call the functions, and print the statistics.

The Code

You'll use the *benford.py* code in this section to investigate voter fraud, but it's flexible enough to be used on *any* dataset where categorical values have been counted, such as medical test results, income tax revenues, or customer refunds. Non-fraud-related applications are also possible, such as detecting process inefficiencies caused by a large number of low-value transactions; problems with data collection and handling, such as missing data, truncated values, or typos; and bias in measurement strategies or surveys, such as favoring best-case or worst-case sampling.

You can download the code from

<https://www.nostarch.com/impracticalpython/>. You'll also need the *Illinois_votes.txt* text file described in “**The Dataset**” on **page 353**.

Importing Modules and Loading Data

Listing 16-1 imports modules and defines a function to load data. For this project, you'll use data in the form of a tab-delimited text file exported from Microsoft Excel, which you load as a list of strings.

benford.py, part 1

```
import sys
import math
❶ from collections import defaultdict
```

```

❷ import matplotlib.pyplot as plt

    # Benford's law percentages for leading digits 1-9
❸ BENFORD = [30.1, 17.6, 12.5, 9.7, 7.9, 6.7, 5.8, 5.1, 4.6]

❹ def load_data(filename):
    """Open a text file & return a list of strings."""
    ❺ with open(filename) as f:
        return f.read().strip().split('\n')

```

Listing 16-1: Imports modules and defines a function for loading data

Most of the imports should be familiar at this point. The `collections` module provides specialized alternatives to standard Python containers like sets, tuples, lists, and dictionaries ❶. For counting first-digit frequencies, you'll need `defaultdict`, which is a `dict` subclass that calls a factory function to supply missing values. With `defaultdict`, you can build a dictionary using a loop, and it will automatically create new keys rather than throwing an error. It returns a dictionary object.

The final import is for plotting with `matplotlib` ❷. For more information on `matplotlib` and how to install it, see “**The Probability-of-Detection Code**” on **page 194**.

Now, assign a variable to a list containing the Benford's law percentages, ordered from 1 to 9 ❸. Then, define a function to read a text file and return a list ❹. As you've done before, use `with`, as it will automatically close the file when done ❺.

Counting First Digits

Listing 16-2 defines a function to count the first digits and store the results in a dictionary data structure. The final counts, and the frequency of each count as a percentage, are returned as lists to use in subsequent functions. The function will also run quality control on the data.

```

❶ def count_first_digits(data_list):
    """Count 1st digits in list of numbers; return counts & frequency."""
    ❷ first_digits = defaultdict(int) # default value of int is 0
    ❸ for sample in data_list:
        ❹ if sample == '':
            continue
        try:
            int(sample)
        except ValueError as e:
            print(e, file=sys.stderr)
            print("Samples must be integers. Exiting", file=sys.stderr)
            sys.exit(1)

        ❺ first_digits[sample[0]] += 1
    # check for missing digits
    keys = [str(digit) for digit in range(1, 10)]
    for key in keys:
        if key not in first_digits:
            first_digits[key] = 0
    ❻ data_count = [v for (k, v) in sorted(first_digits.items())]
    total_count = sum(data_count)
    data_pct = [(i / total_count) * 100 for i in data_count]
    ❼ return data_count, data_pct, total_count

```

Listing 16-2: Defines a function to count first digits and return counts and frequency

The `count_first_digits()` function takes the list of strings returned from the `load_data()` function as an argument ❶. You'll call it in `main()`.

Start a dictionary, named `first_digits`, using `defaultdict` ❷. This step just sets up the dictionary for later population. The first argument for `defaultdict` is any callable (with no arguments). In this case, the callable is the type constructor for `int`, as you want to count integers. With `defaultdict`, whenever an operation encounters a missing key, a function named `default_factory` is called with no arguments, and the output is used as the value. Nonexistent keys get the value returned by `default_factory`.

Now start a `for` loop and go through the samples in `data_list` ❸. If the sample is empty—that is, if the text file contains a blank line ❹—skip it with `continue`. Otherwise, use `try` to convert the sample to an integer. If an exception occurs, the sample isn’t a proper count value, so let the user know and exit the program. In the following output example, the input file contains a float value (`0.01`) and the `main()` function prints the filename.

```
Name of file with COUNT data: bad_data.txt
invalid literal for int() with base 10: '0.01'
Samples must be integers. Exiting.
```

If the sample passes the test, make its first element (leading digit) the dictionary key and add 1 to the value ❺. Because you used `defaultdict` with `int`, the key is initially assigned a default value of 0 on the fly.

To compare the counts to a Benford’s law distribution, you need the keys listed in numerical order, so use list comprehension and `sorted` to make a new version of `first_digits`, named `data_count` ❻. This will yield the values sorted by key, as shown here:

```
[129, 62, 45, 48, 40, 25, 23, 21, 15]
```

Next, sum the counts, then make a new list and convert the counts to percentages. End the function by returning these two lists and the summed counts ❼. Because the counts in the lists are sorted from 1 to 9, you don’t need the associated first digit—it is implicit in the ordering.

Getting the Expected Counts

Listing 16-3 defines the `get_expected_counts()` function that takes the observed data and calculates what the expected counts should be for the leading digits, based on Benford’s law. These expected counts are returned as a list that you’ll use later with the chi-square goodness-of-fit test to see how well the observed data conform to Benford’s law.

```

❶ def get_expected_counts(total_count):
    """Return list of expected Benford's law counts for a total sample count."""
    ❷ return [round(p * total_count / 100) for p in BENFORD]

```

Listing 16-3: Defines a function to calculate expected Benford's law counts for a dataset

The argument for this function is the summed count that you returned from the `count_first_digits()` function in **Listing 16-2** ❶. To get the counts you would expect for Benford's law, you'll need to use the frequency *probability* of each digit, so convert the percentages in the `BENFORD` list by dividing by 100. Then multiply the `total_count` variable by this probability. You can do all this with list comprehension as part of the `return` statement ❷.

Determining Goodness of Fit

Listing 16-4 defines a function to implement the chi-square test described in “**Performing the Chi-Square Test**” on **page 352**. This test calculates the goodness of fit of the observed counts to the expected counts predicted by Benford's law. The function will first calculate the chi-square test statistic, then compare it to the chi-square distribution table entry at a *p*-value of 0.05 for 8 degrees of freedom. Based on the comparison, the function returns either `True` OR `False`.

benford.py, part 4

```

❶ def chi_square_test(data_count, expected_counts):
    """Return boolean on chi-square test (8 degrees of freedom & P-val=0.05)."""
    ❷ chi_square_stat = 0 # chi-square test statistic
    ❸ for data, expected in zip(data_count, expected_counts):
        ❹ chi_square = math.pow(data - expected, 2)
        chi_square_stat += chi_square / expected
    ❺ print("\nChi Squared Test Statistic = {:.3f}".format(chi_square_stat))
    print("Critical value at a P-value of 0.05 is 15.51.")

```

```
⑥ return chi_square_stat < 15.51
```

Listing 16-4: Defines a function to measure goodness of fit of observed data versus Benford's law

The chi-square test works on counts, so the function needs the lists of data counts and expected counts that the `count_first_digits()` and `get_expected_counts()` functions returned ①. Assign a variable, named `chi_square_stat`, to hold the chi-square test statistic, and assign it a value of 0 ②.

Use `zip` to loop through the nine values in `data_count` and `expected_counts`; `zip` will pair the first item in one list with the first item in the second list, and so on ③. To calculate the chi-square statistic, first subtract the counts for each digit and square the result ④. Then, divide this value by the expected count for the digit and add the result to the `chi_square_stat` variable. Then print the result to three decimal places ⑤.

Return the boolean test for the `chi_square_stat` variable versus 15.51, which is the critical value corresponding to a p -value of 0.05 at 8 degrees of freedom (see **Table 16-1**) ⑥. If `chi_square_stat` is less than this value, the function will return `True`; otherwise, it will return `False`.

Defining the Bar Chart Function

Listing 16-5 defines the first part of a function to display the observed count percentages as a `matplotlib` bar chart. You used similar code in **Chapter 12** to graph the outcomes of the retirement nest-egg simulations. This function will also plot the Benford's law percentages as red dots, so you can visually estimate how well the observed data fit the expected distribution.

The `matplotlib` website contains many code examples for building a wide variety of plots. This code is partly based on the demo example at https://matplotlib.org/examples/api/barchart_demo.html.

```

❶ def bar_chart(data_pct):
    """Make bar chart of observed vs expected 1st-digit frequency (%)."""
    ❷ fig, ax = plt.subplots()

    ❸ index = [i + 1 for i in range(len(data_pct))] # 1st digits for x-axis

    # text for labels, title, and ticks
    ❹ fig.canvas.set_window_title('Percentage First Digits')
    ❺ ax.set_title('Data vs. Benford Values', fontsize=15)
    ❻ ax.set_ylabel('Frequency (%)', fontsize=16)
    ❼ ax.set_xticks(index)
    ax.set_xticklabels(index, fontsize=14)

```

Listing 16-5: Defines the first part of the `bar_chart()` function

Define the `bar_chart()` function that takes as an argument the list of frequencies—as percentages—of the first digits in the observed data ❶. The `plt.subplots()` function returns a tuple of figure and axes objects; unpack this tuple into variables named `fig` and `ax` ❷.

Next, use list comprehension to make a list of digits from 1 to 9 ❸. This `index` variable will define the x-axis location of each of the nine vertical bars in the chart.

Set up the plot’s title, labels, and so on. Name the plot *window* 'Percentage First Digits' ❹, then display a title *within* the plot ❺. I am using generic titles, but you can customize these to be more specific. Use the `fontsize` keyword argument to set the text size to 15. Note that the window title is an attribute of `fig`, but the other labels will be attributes of `ax`.

Use `set_ylabel()` to name the y-axis “Frequency (%)” ❻, then set the x-axis tick marks based on the `index` variable ❼. The tick labels will be the numbers 1 through 9, so use the `index` variable again and set the font size to 14.

Completing the Bar Chart Function

Listing 16-6 completes the `bar_chart()` function by defining the bars, annotating the top of each bar with its frequency value, and plotting the Benford distribution values as red-filled circles.

benford.py, part 6

```
# build bars
❶ rects = ax.bar(index, data_pct, width=0.95, color='black', label='Data')

# attach a text label above each bar displaying its height
❷ for rect in rects:
    ❸ height = rect.get_height()
    ❹ ax.text(rect.get_x() + rect.get_width()/2, height,
              '{:0.1f}'.format(height), ha='center', va='bottom',
              fontsize=13)

# plot Benford values as red dots
❺ ax.scatter(index, BENFORD, s=150, c='red', zorder=2, label='Benford')

# Hide the right and top spines & add legend
❻ ax.spines['right'].set_visible(False)
   ax.spines['top'].set_visible(False)
❼ ax.legend(prop={'size':15}, frameon=False)

❽ plt.show()
```

Listing 16-6: Completes the function for generating a bar chart

Name a variable, called `rects`, for rectangles, and use it to hold the bars for the bar chart ❶. You generate these with the `bar()` method, which returns a container with all the bars. Pass it the `index` variable and list of frequency counts as percentages, set the width of each bar to 0.95, fill them with black, and set the `label` argument to `'Data'`. The last parameter is a very handy way to autogenerate a legend. You'll take advantage of this near the end of the function.

I'm a fan of plotting the actual bar value just above the bar so you don't have to squint over at the y-axis and try to guess it. To do this, start by

looping through each bar (`rect`) in `rects` ❷ and get its height ❸, which is its y-axis value. Then, call the `ax` object's `text()` method ❹ and pass it the x-location of the left-hand side of the bar—obtained with the `get_x()` method—and add to it half the width of the bar in order to center the label over the bar. Because you use the `get_width()` method, you only have to assign the bar width once, which you did in step ❶. Next comes the bar height—formatted to one decimal place—followed by the horizontal and vertical alignments. Set these to the center and the bottom of the text bounding box, respectively. Finish by assigning the text size.

Now, start building the `matplotlib` “markers”—in this case, dots—that will flag the location of the Benford distribution frequencies for each first digit. Do this using the `scatter()` method, which builds scatterplots ❺.

The first two arguments for `scatter()` are the x-y locations for each marker, represented by consecutive pairs from the `index` and `BENFORD` lists. Next is the marker size, set at 150, followed by a color. Both `red` and `DodgerBlue` work well. You want the markers to post on top of the bars, so set the `zorder` to 2. Elements in the figure are referred to as `matplotlib` “artists,” and artists with higher `zorder` values will plot over those with lower values. Finish with the `label` argument used to make a legend.

The next two statements are for aesthetics. By default, `matplotlib` will draw a box around the interior of the plot, and the upper border may interfere with the labels posted at the top of each bar. So, remove the top and right borders by setting their visibility to `False` ❻.

Use `legend()` to build a legend for the plot ❼. This will work with no arguments, but set its size attribute to 15 and turn off the frame around the legend for an arguably more attractive result. End by calling `plt.show()` to display the chart ❽. An example bar chart is shown in **Figure 16-7**.

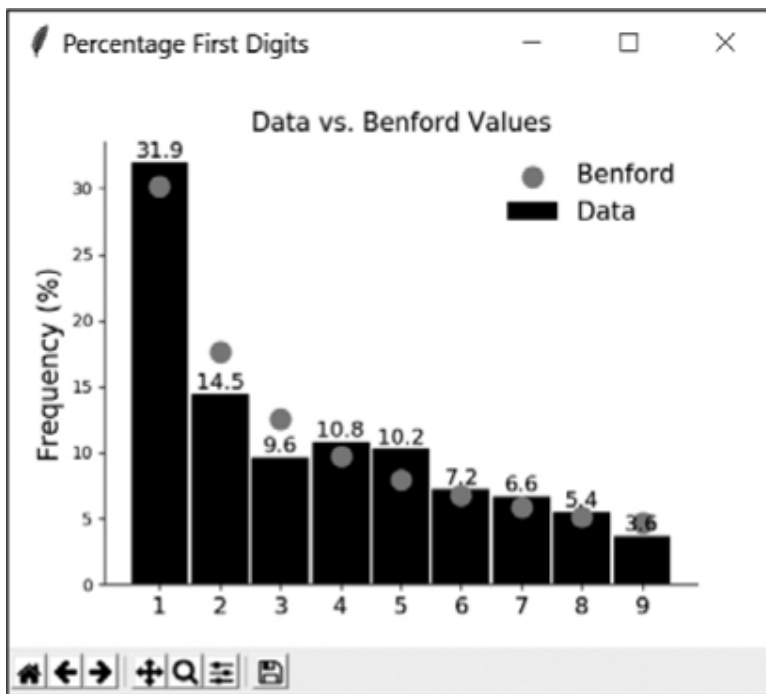


Figure 16-7: Example output of the `bar_chart()` function

In the `main()` function, you will display additional information as text in the interpreter window. This will include the value of the chi-square test statistic.

Defining and Running the `main()` Function

Listing 16-7 defines the `main()` function and runs the program as a module or stand-alone. Since most of the work is done in the individual functions, `main()` “mainly” calls these functions and prints some statistics.

benford.py, part 7

```
def main():
    """Call functions and print stats."""
    # load data
    while True:
        ❶ filename = input("\nName of file with COUNT data: ")
        try:
            data_list = load_data(filename)
        except IOError as e:
            print("{} Try again.".format(e), file=sys.stderr)
        else:
            break
```

```

❷ data_count, data_pct, total_count = count_first_digits(data_list)
❸ expected_counts = get_expected_counts(total_count)
   print("\nobserved counts = {}".format(data_count))
   print("expected counts = {}".format(expected_counts), "\n")

❹ print("First Digit Probabilities:")
❺ for i in range(1, 10):
    print("{}: observed: {:.3f}  expected: {:.3f}".
          format(i, data_pct[i - 1] / 100, BENFORD[i - 1] / 100))

❻ if chi_square_test(data_count, expected_counts):
    print("Observed distribution matches expected distribution.")
else:
    print("Observed distribution does not match expected.",
          file=sys.stderr)

❼ bar_chart(data_pct)

❽ if __name__ == '__main__':
    main()

```

Listing 16-7: Defines the `main()` function and runs the program as a module or stand-alone

Start by asking the user to input the name of the file with the count data to be analyzed ❶; embed this request in a `while` loop that will continue until the user enters a valid filename or closes the window. The user can enter a filename or a full pathname if they want to load a dataset that isn't stored in the current working directory. For example, in Windows:

```
Name of file with COUNT data: C:\Python35\Benford\Illinois_votes.txt
```

Use a `try` statement to call the `load_data()` function you built earlier and pass the function the filename. If the filename is valid, the returned list is assigned to the `data_list` variable. If an exception occurs, catch it and print the error. Otherwise, `break` from the `while` loop.

Next, pass the returned list of data counts to the `count_first_digits()` function and unpack the results as the variables `data_count`, `data_pct`, and `total_count`, which are lists of the first-digit counts, percentages, and total number of counts, respectively ❷. Then, generate a list of the counts expected for a Benford's law distribution by calling the `get_expected_counts()` function and passing it the `total_count` variable ❸. Print the lists of observed and expected counts.

Now, make a table that compares the first-digit frequency in the data with the expected values. Use probabilities, as decimal values are easy to keep attractively aligned in the shell. Start with a header `print` statement ❹, then loop through the numbers 1 to 9, printing the results for the observed counts (data), followed by the expected counts, each to three decimal places ❺. Note that the indexes in the two lists start with zero, so you have to subtract 1 from `i`.

Pass the two count lists to the `chi_square_test()` function in order to calculate how well the observed data fit the expected distribution ❻. If the function returns `True`, use a `print` statement to let the user know the observed distribution matches Benford's law (or, more technically, *there is no significant difference* between the two). Otherwise, report that they don't match, and for shell users, color the font red.

The `chi_square_test()` function will display its results in the interpreter window, so call the `bar_chart()` function to generate a bar chart ❼. Pass it the list of data counts as percentages.

Back in the global space, end the program with the code for running it as a module or in stand-alone mode ❽.

If you run the program on the *Illinois_votes.txt* dataset, you will see the output shown in **Figure 16-8**. There is no apparent anomaly in the voting results, based on Benford's law.

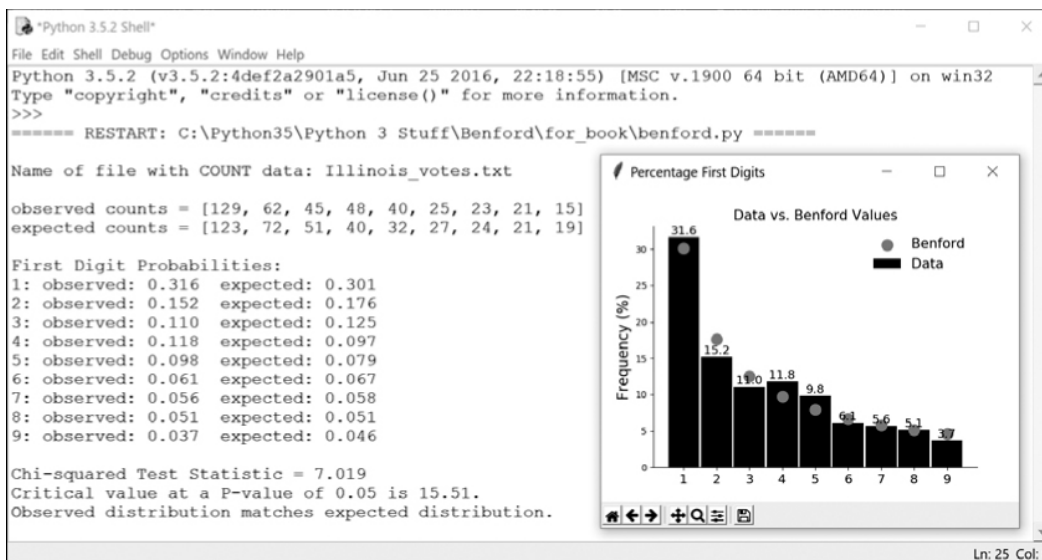


Figure 16-8: Output of benford.py for the dataset Illinois_votes.txt

If you run the program using only the Trump votes, then only the Clinton votes, you get the results shown in **Figure 16-9**. Trump's distribution, with a test statistic of 15.129, barely passes the chi-square test.

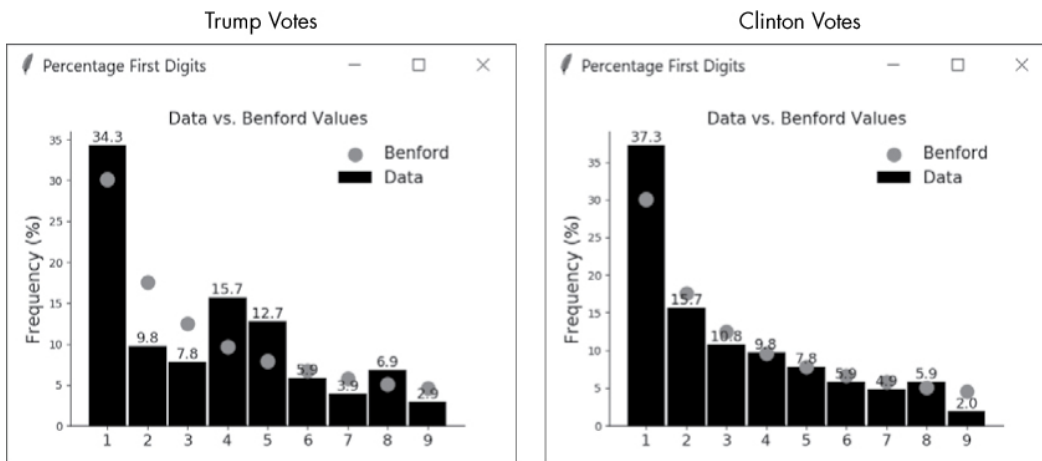


Figure 16-9: Comparison of Trump results (left) versus Clinton results (right) for the state of Illinois

In cases like this, you should be careful about drawing immediate conclusions. The dataset is small—only 102 samples per candidate—and the results may be influenced by things like demographic and voter turnout differences between rural and urban areas. An interesting article on this urban-rural divide can be found at

<http://www.chicagotribune.com/news/data/ct-illinois-election-urban-rural-divide-2016-htmlstory.html>.

In **“Practice Project: Beating Benford”** on **page 364**, you'll get a chance to tamper with the Illinois vote counts and alter the outcome. You'll then

use the preceding code to see how well the results conform to Benford's law.

Summary

Way back in **Chapter 1**, we used the “**Poor Man's Bar Chart**” practice project on **page 15** and the “**Poor Foreign Man's Bar Chart**” challenge project on **page 16** to look at how the frequency of occurrence of letters in a language is both irregular and predictable. This provides a powerful tool for cryptanalysis. Here at the end of the book, we've come full circle and found that even numbers share this trait, resulting in a powerful tool for fraud detection. With a short and simple Python program, you can shake the very pillars of heaven and bring the high and mighty crashing down to earth—all because someone noticed that the front pages of a book were dirty.

Well, that does it for *Impractical Python Projects*. I hope you had fun, learned something new, and were inspired to create your own set of impractical projects!

Further Reading

Benford's Law: Applications for Forensic Accounting, Auditing, and Fraud Detection (John Wiley & Sons, 2012) by Mark Nigrini covers the mathematics, theory, and testing of Benford's law together with example applications, including fraud, tax evasion, and Ponzi schemes.

Practice Project: Beating Benford

Test your skill at rigging an election using this practice project. You can find a solution, *beat_benford_practice.py*, in the appendix or download it from <https://www.nostarch.com/impracticalpython/>.

A dataset shouldn't be considered valid just because it follows Benford's law. The reason is simple: if you know about Benford's law, then you can beat it.

To prove this, pretend you're a high-level hacker for a nefarious foreign government with access to all the voting records in the state of Illinois. Write a Python program that tampers with county-wide votes so that

Donald Trump wins the state, but the vote counts still obey Benford's law. Be careful; Illinois is a "blue" state, so you don't want to engineer a landslide victory (loosely defined as a 10–15 percentage point advantage in the popular vote). To avoid suspicion, Trump should squeak by with a few percentage points.

NOTE

States have rules regarding the recount of votes. Before manipulating an election, a fraudster would want to be aware of these to avoid the scrutiny a recount would bring. The actual statutory rules for each state are no fun to read, but the Citizens for Election Integrity Minnesota provides approachable summaries. The one for Illinois can be found at <https://ceimn.org/searchable-databases/recount-database/illinois/>.

Your program should steal votes from the other candidates, while preserving the by-county totals; that way, the total number of votes cast doesn't change. As a quality-control step, print out the old and new vote totals by county for Trump and Clinton, as well as their old and new statewide totals. Then, write out a text file that you can input into *benford.py* so you can check how you did with respect to Benford's law.

Datasets for each candidate are already prepared and listed here; you can download them from <https://www.nostarch.com/impracticalpython/>. Each of these datasets is just a column of numbers, representing votes, that has been sorted alphabetically by county (so don't change the order!).

Clinton_votes_Illinois.txt

Johnson_votes_Illinois.txt

Stein_votes_Illinois.txt

Trump_votes_Illinois.txt

Figure 16-10 shows the results of running *benford.py* on the output from my attempt, *beat_benford_practice.py*, which used the preceding datasets. The distribution passes the chi-square test and yields a visually convincing—but plausibly imperfect—fit to the values predicted by Benford’s law.

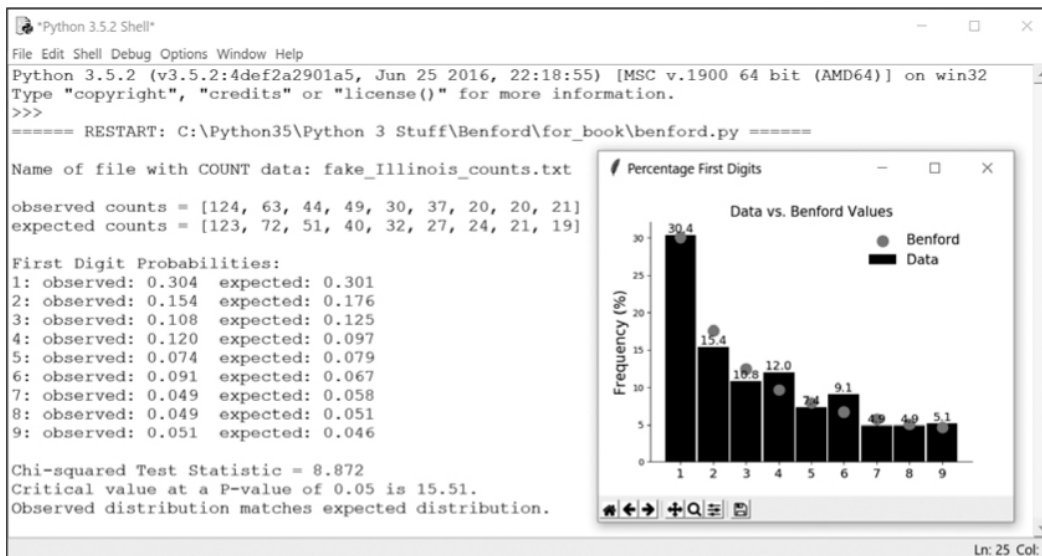


Figure 16-10: Results of running the distribution from beat_benford_practice.py in benford.py. Mischief managed!

Shown here are a few lines of output from *beat_benford_practice.py*, with the old and new vote totals by county:

old Trump: 12412	new Trump: 13223	old Clinton: 3945	new Clinton: 3290
old Trump: 13003	new Trump: 14435	old Clinton: 7309	new Clinton: 6096
old Trump: 453287	new Trump: 735863	old Clinton: 1611946	new Clinton: 1344496
old Trump: 6277	new Trump: 6674	old Clinton: 1992	new Clinton: 1661
old Trump: 4206	new Trump: 4426	old Clinton: 1031	new Clinton: 859

The third line from the top represents Cook County, which contains Chicago. Note that Clinton still wins here, but by a lower margin. For Trump to win this blue county outright would be a tremendous red flag that vote tampering had occurred, even if he won the whole state by only a small margin!

Challenge Projects

Try your hand at these challenge projects. No solutions are provided.

Benfording the Battlegrounds

No candidate needs to cheat in a state they're guaranteed to win. If you're an investigator looking into voter fraud, you would most likely start with the battleground states. These could swing either way, and candidates spend much of their war chest—and time—on these places. According to Ballotpedia (<https://ballotpedia.org>), Trump's battleground states in 2016 were Arizona, Iowa, Michigan, Wisconsin, Ohio, Pennsylvania, North Carolina, and Florida. Clinton's were Colorado, Nevada, New Hampshire, and Virginia.

The online voting records for states are commonly provided in a number of formats, such as Microsoft Excel spreadsheets. Gather the ones for the battleground states, convert them to text files, and run them through *benford.py*. To get you started, you can find the Ohio records here: <https://www.sos.state.oh.us/elections/>.

While No One Was Looking

Former Speaker of the US House of Representatives Tip O'Neill was famous for saying, "All politics is local." Take this to heart and use the *benford.py* program to check out some of your local area races, such as for judges, mayors, county supervisors, sheriffs, and city council members. These events generally attract less scrutiny than do races for Senate seats, governor, or president. If you find an irregularity, make sure the voting dataset is a valid application for Benford's law before raising a stink!